

Structural testing and code coverage

This chapter covers

- Creating test cases based on the code structure
- Combining structural testing and specification-based testing
- Using code coverage properly
- Why some developers (wrongly) dislike code coverage

In the previous chapter, we discussed using software requirements as the main element to guide the testing. Once specification-based testing is done, the next step is to *augment the test suite with the help of the source code*. There are several reasons to do so.

First, you may have forgotten a partition or two when analyzing the requirements, and you may notice that while looking at the source code. Second, when implementing code, you take advantage of language constructs, algorithms, and data structures that are not explicit in the documentation. Implementation-specific details should also be exercised to increase the likelihood of ensuring the program's full correctness.

In this chapter, we learn how to systematically reflect on the source code, see what is being exercised by the test suite we derived with the help of the specification, and what remains to be tested. Using the structure of the source code to guide testing is also known as *structural testing*. Understanding structural testing techniques means understanding the *coverage criteria*. The remainder of this chapter explores using code coverage information to gain more confidence that the program works as expected.

3.1 Code coverage, the right way

Consider the following requirement for a small program that counts the number of words in a string that end with either “r” or “s” (inspired by a CodingBat problem, <https://codingbat.com/prob/p199171>):

Given a sentence, the program should count the number of words that end with either “s” or “r”. A word ends when a non-letter appears. The program returns the number of words.

A developer implements this requirement as shown in the following listing.

Listing 3.1 Implementing the CountWords program

```
public class CountWords {
    public int count(String str) {
        int words = 0;
        char last = ' ';

        for (int i = 0; i < str.length(); i++) {

            if (!isLetter(str.charAt(i)) &&
                (last == 's' || last == 'r')) {
                words++;
            }

            last = str.charAt(i);

            if (last == 'r' || last == 's') {
                words++;
            }

            return words;
        }
    }
}
```

Loops through each character in the string

If the current character is a non-letter and the previous character was “s” or “r”, we have a word!

Stores the current character as the “last” one

Counts one more word if the string ends in “r” or “s”

Now, consider a developer who does not know much about specification-based testing techniques and writes the following two JUnit tests for the implementation.

Listing 3.2 Initial (incomplete) tests for CountWords

```

@Test
void twoWordsEndingWithS() {
    int words = new CountLetters().count("dogs cats");
    assertThat(words).isEqualTo(2);
}

@Test
void noWordsAtAll() {
    int words = new CountLetters().count("dog cat");
    assertThat(words).isEqualTo(0);
}

```

Two words ending in “s” (dogs and cats): we expect the program to return 2.

No words ending in “s” or “r” in the string: the program returns 0.

This test suite is far from complete—for example, it does not exercise words ending in “r”. Structural testing shows its value in such situations: we can identify parts of the test code that our test suite does not exercise, determine why this is the case, and create new test cases.

Identifying which parts of the code our tests exercise is straightforward today, thanks to the many production-ready code coverage tools on the market for all programming languages and environments. For example, figure 3.1 shows the report generated by JaCoCo (www.jacoco.org/jacoco), a very popular code coverage tool for Java, after running the two tests in listing 3.2.

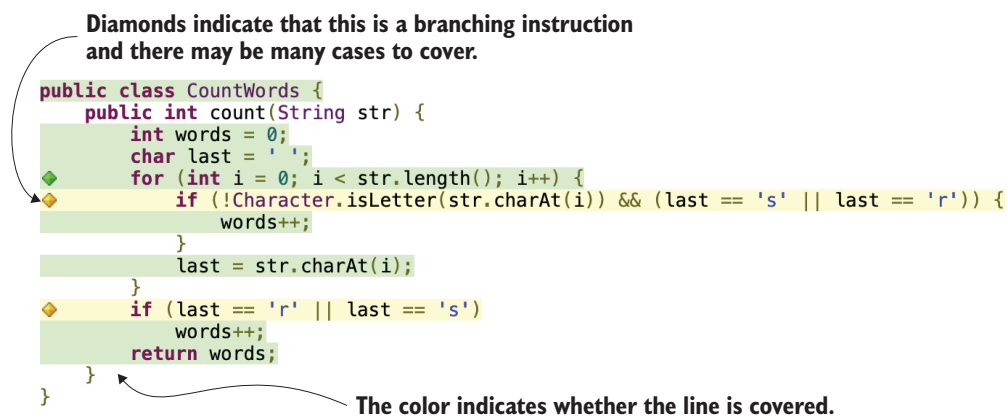


Figure 3.1 Code coverage achieved by the two tests in the `CountWords` implementation. The two `if` lines are only partially covered.

The background color of each line indicates its coverage (the colors appear as shades of gray in the printed book):

- A green background indicates that a line is completely covered by the test suite. In the figure, all lines with the exception of the two `if`s are green.
- A yellow background means the line is partially covered by the test suite. For example, in the figure, the two `if` statement lines are only partially covered.

- A red background means the line is not covered. In the figure, there are no red lines, which means all lines are exercised by at least one test.
- Lines with no background color (such as `}`) are lines the coverage tool does not see. Behind the scenes, coverage tools are instrumenting the compiled byte-code of the program. Things like closing brackets and method declaration lines are not really counted.

JaCoCo also uses a diamond to identify a line that may branch the program, including the `for` and `if` statements in figure 3.1, as well as `while`, `for`, `do-while`, ternary ifs, lambda expressions, and so on. Hovering your mouse over the diamond shows the details.

As previously mentioned, the first `if` statement has a yellow background, indicating that although the line is covered, not all of its branches are. When I look at the details of the report, the tool says that one out of six combinations (three conditions in the `if` statement times two options, `true` and `false`) is not covered. See figure 3.2.

```

1. package book;
2.
3. public class CountWords {
4.     public int count(String str) {
5.         int words = 0;
6.         char last = ' ';
7.          for (int i = 0; i < str.length(); i++) {
8.              if (!Character.isLetter(str.charAt(i)) && (last == 's' || last == 'r')) {
9.                 words++;
10.            }
11.            last = str.charAt(i);
12.        }
13.         if (last == 'r' || last == 's')
14.            words++;
15.        return words;
16.    }
17. }

```

Figure 3.2 JaCoCo shows how many branches we are missing.

The current test suite does not fully exercise the `last == 'r'` condition. This is useful information; thanks to structural testing, the tester can now figure out why this test case did not emerge before.

Reasons to miss a test case

Here are some pragmatic reasons a developer may miss a test case:

- The developer made a mistake. The specification was clear about the requirement.
- The specification did not mention the case, and it is unclear whether the behavior is expected. The developer must decide whether to bring it to the requirements engineer. Is it a mistake in the implementation?

- The specification did not mention the case, but the code has a reason to exist. For example, implementation details such as performance and persistence often force developers to write code that is not reflected in the (functional) requirement. The developer should add a new test to the test suite, which will exercise the implementation-specific behavior that may cause bugs.

Moving on with the example, we write a test case that exercises the “words that end in ‘r’” partition as follows.

Listing 3.3 Testing for words that end in “r”

```
@Test
void wordsThatEndInR() {
    int words = new CountWords().count("car bar");
    assertThat(words).isEqualTo(2);
}
```

Words that end in “r”
should be counted.

With the newly added test case in the test suite, we rerun the coverage tool. Figure 3.3 shows the new JaCoCo report. Every line is now fully covered: we have covered all the lines and conditions of the code under test. If parts of the code were still not covered, we would repeat the process: identify uncovered parts, understand why they are not covered, and write a test that exercises that piece of code.

All lines are green, which means all lines and branches of the method are covered by at least one test case.

```
public class CountWords {
    public int count(String str) {
        int words = 0;
        char last = ' ';
        for (int i = 0; i < str.length(); i++) {
            if (!Character.isLetter(str.charAt(i)) && (last == 's' || last == 'r')) {
                words++;
            }
            last = str.charAt(i);
        }
        if (last == 'r' || last == 's')
            words++;
        return words;
    }
}
```

Figure 3.3 Code coverage of the three tests for the `CountWords` implementation. The test suite now achieves full coverage of branches and conditions.

3.2 *Structural testing in a nutshell*

Based on what we just did, let me define a simple approach that any developer can follow (see figure 3.4):

- 1 Perform *specification-based testing*, as discussed in the previous chapter.
- 2 *Read the implementation*, and understand the main coding decisions made by the developer.
- 3 Run the devised test suite with a *code coverage tool*.
- 4 For each piece of code that is *not* covered:
 - a *Understand* why that piece of code was not tested. Why didn't you see this test case during specification-based testing? Consult with the requirements engineer if you need more clarity.
 - b *Decide* whether the piece of code deserves a test. Testing or not testing that piece of code is now a conscious decision on your part.
 - c If a test is needed, *implement an automated test case* that covers the missing piece.
- 5 Go back to the source code and *look for other interesting tests you can devise* based on the code. For each identified piece of the code, perform the substeps of step 4.

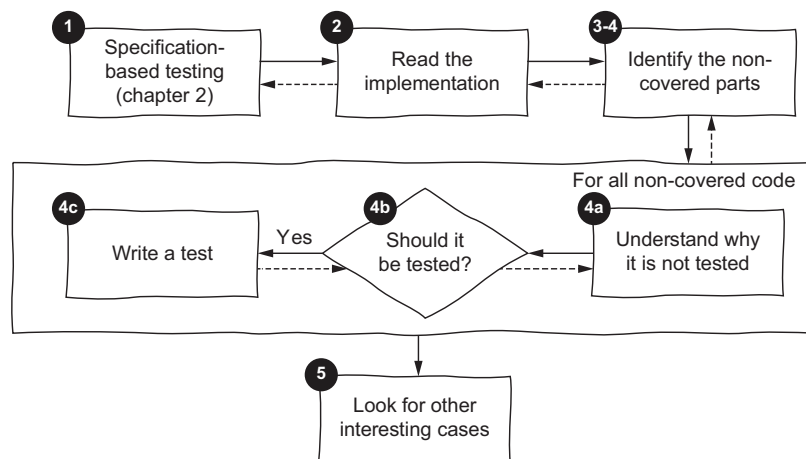


Figure 3.4 Applying structural testing in a nutshell. Arrows indicate the iterative nature of the process. The diamond represents the moment where the developer decides whether to write the test case.

The most important thing about this approach is that *structural testing complements the test suite previously devised via specification-based testing*. The code coverage tool is an automated way to identify parts that are not covered.

Just like the approach I proposed in chapter 2, this one is meant to be iterative and not to restrain you to a single way of working. It is not uncommon to go back to the specification and devise additional interesting test cases.

Before I show another running example of structural testing and discuss how to pragmatically use it in our daily lives, the next section introduces the coverage criteria we use with this approach.

3.3 Code coverage criteria

Whenever we identify a line of code that is not covered, we have to decide how thorough (or rigorous) we want to be when covering that line. Let's revisit an `if` statement from the `CountWords` program.

Listing 3.4 An `if` expression from the `CountWords` program

```
if (!Character.isLetter(str.charAt(i)) &&
    (last == 's' || last == 'r'))
```

A developer may decide to only *cover the line*—in other words, if a test passes through that `if` line, the developer will consider it covered. A single test case can do this. A slightly more thorough developer may cover the `if` being evaluated to `true` and `false`; doing so requires two test cases. A third developer may explore each condition in the `if` statement. This particular `if` has three conditions requiring at least two tests each, for a total of six tests. Finally, a very thorough tester may decide to cover every possible execution path of this statement. Given that it has three different conditions, doing so requires $2 \times 2 \times 2 = 8$ test cases.

Let's formalize this discussion. Note that you've already seen some of these terms.

3.3.1 Line coverage

A developer who aims to achieve line coverage wants at least one test case that covers the line under test. It does not matter if that line contains a complex `if` statement full of conditions. If a test touches that line in any way, the developer can count the line as covered.

3.3.2 Branch coverage

Branch coverage takes into consideration the fact that branching instructions (`ifs`, `fors`, `whiles`, and so on) make the program behave in different ways, depending how the instruction is evaluated. For a simple `if (a && b)` statement, having a test case T1 that makes the `if` statement `true` and another test case T2 that makes the statement `false` is enough to consider the branch covered.

Figure 3.5 illustrates a control-flow graph (CFG) of the `CountWords` program. You can see that for each `if` instruction, two edges come out of the node: one representing where the flow goes if the statement is evaluated to `true` and another representing where the program goes if the statement is evaluated to `false`. Covering all the edges in the graph means achieving 100% branch coverage.

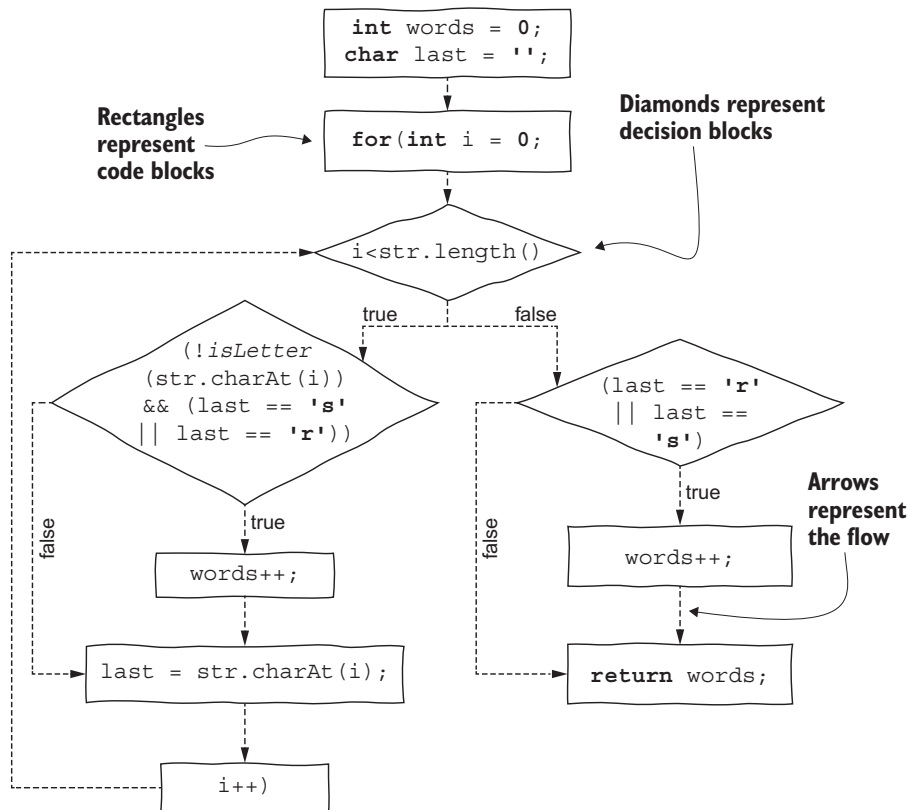


Figure 3.5 A control-flow graph of the CountWords program

3.3.3 Condition + branch coverage

Condition + branch coverage considers not only possible branches but also each condition of each branch statement. For example, the first if statement in the CountWords program contains three conditions: `!Character.isLetter(str.charAt(i))`, `last == 's'`, and `last == 'r'`. Therefore, a developer aiming for condition + branch coverage should create a test suite that exercises each of those individual conditions being evaluated to true and false at least once *and* the entire branch statement being true and false at least once.

Note that blindly looking only at the conditions (and ignoring how they are combined) may result in test suites that do not cover everything. Imagine a simple `if (A || B)`. A test suite composed of two tests (T1 that makes A true and B false and T2 that makes A false and B true) covers the two conditions, as each condition is exercised as true and false. However, the test suite does not fully cover the branch, as in both tests, the evaluation of the entire if statement is always true. This is why we use condition + branch coverage, and not only (basic) condition coverage.

In the extended CFG in figure 3.6, branch nodes contain only a single condition. The complicated if is broken into three nodes.

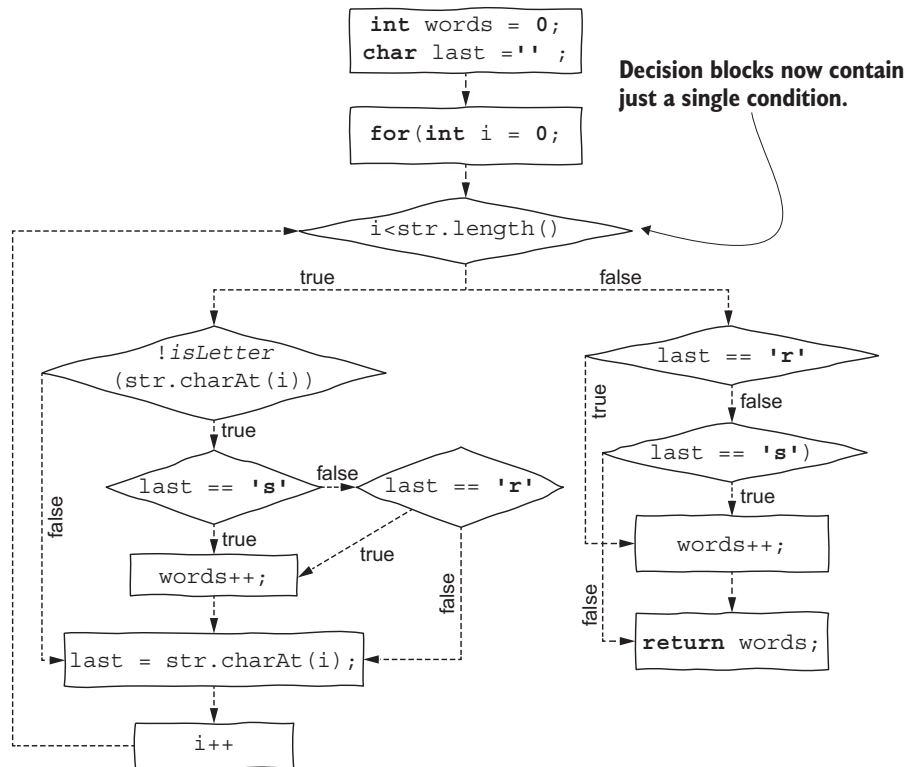


Figure 3.6 The extended control-flow graph of the CountWords program. Each condition is in its own node. Covering all the edges in the graph means achieving 100% condition + branch coverage.

3.3.4 Path coverage

A developer aiming for path coverage covers *all* the possible paths of execution of the program. While ideally this is the strongest criterion, it is often impossible or too expensive to achieve. In a single program with three conditions, where each condition could be independently evaluated to true or false, we would have $2^3 = 8$ paths to cover. In a program with 10 conditions, the total number of combinations would be $2^{10} = 1024$. In other words, we would need to devise more than a thousand tests!

Path coverage also gets more complicated for programs with loops. In a program with an unbounded loop, the loop might iterate hundreds of times. A rigorous tester aiming for path coverage would have to try the program with the loop executing one time, two times, three times, and so on.

3.4 Complex conditions and the MC/DC coverage criterion

Devising test suites that maximize the number of bugs they can identify while minimizing the effort/cost of building the test suite is part of any tester's job. The question is, what can we do about complex, lengthy if statements? Modified condition/decision coverage (MC/DC) is a good answer.

The MC/DC criterion looks at combinations of conditions, as path coverage does. However, instead of testing *all* possible combinations, we identify the *important* combinations that need to be tested. MC/DC exercises each of these conditions so that it can, independently of the other conditions, affect the outcome of the entire decision. Every possible condition of each parameter must influence the outcome at least once. (For details, read Kelly Hayhurst's 2001 paper.)

3.4.1 An abstract example

Let's take a simple abstract example: `if (A && (B || C))`, where A, B, and C all evaluate to booleans. MC/DC dictates the following:

- For condition A:
 - There must be one test case where A = true (say, T1).
 - There must be one test case where A = false (say, T2).
 - T1 and T2 (which we call *independence pairs*) must have different outcomes (for example, T1 makes the entire decision evaluate to true, and T2 makes the entire decision evaluate to false).
 - Variables B and C in T1 must be equivalent (either both evaluate to true or both evaluate to false) to B and C in T2. In other words, B and C must have the same truth values in T1 and T2.
- For condition B:
 - There must be one test case where B = true (say, T3).
 - There must be one test case where B = false (say, T4).
 - T3 and T4 must have different outcomes.
 - Variables A and C in T3 must be equivalent to A and C in T4.
- For condition C:
 - There must be one test case where C = true (say, T5).
 - There must be one test case where C = false (say, T6).
 - T5 and T6 have different outcomes.
 - Variables A and B in T5 must be equivalent to A and B in T6.

If conditions have only binary outcomes (that is, true or false), the number of tests required to achieve 100% MC/DC coverage is $N + 1$, where N is the number of conditions in the decision (as shown by Chilenski [2001]). Note that $N + 1$ is smaller than the total number of possible combinations (2^N). So, to devise a test suite that achieves 100% MC/DC, we must create $N + 1$ test cases that, when combined, exercise all the combinations independently from the others.

3.4.2 Creating a test suite that achieves MC/DC

The question is how to (mechanically) select such test cases. Let's continue using the same `if` statement from the `CountWords` program (from listing 3.4). The statement takes three booleans as input: (1) whether the current character is a letter and whether this letter is (2) "s" or (3) "r". Generically, this is the same as the `A && (B || C)` example we just discussed.

To test this program, we first use a truth table to see all the combinations and their outcomes. In this case, we have three decisions, and $2^3 = 8$. Therefore, we have tests T1 to T8, as listed in table 3.1.

Table 3.1 Truth table for the `if` expression from the `CountWords` program

Test case	<code>isLetter</code>	<code>last == s</code>	<code>last == r</code>	decision
T1	true	true	true	true
T2	true	true	false	true
T3	true	false	true	true
T4	true	false	false	false
T5	false	true	true	false
T6	false	true	false	false
T7	false	false	true	false
T8	false	false	false	false

Our goal is to apply the MC/DC criterion to these test cases and select $N + 1$ tests, which in this case means $3 + 1 = 4$. To determine which four tests satisfy MC/DC, we need to go condition by condition, beginning by selecting the pairs of combinations (or tests) for the `isLetter` part of the condition:

- For T1, `isLetter`, `last == s`, and `last == r` are all true, and decision (that is, the outcome of the entire boolean expression) is also true. We now look for another test in the table where the value of `isLetter` is the opposite of the value in T1 but the other values (`last == s` and `last == r`) are the same. This means look for a test where `isLetter` is false, `last == s` is true, `last == r` is true, and decision is false. This combination appears in T5.

Thus, we have found a pair of tests, T1 and T5 (an independence pair), where `isLetter` is the only parameter that is different and the outcome (decision) changes. In other words, for this pair of tests, `isLetter` *independently* influences the outcome (decision). Let's keep the pair {T1, T5} in our list of test cases.

- We could stop here and move to the next variable. But finding all independence pairs for `isLetter` may help us reduce the final number of test cases, as you will see. So let's continue and look at the next test. In T2, `isLetter` is true,

`last == s` is true, `last == r` is false, and decision is true. We repeat the process and search for a test where `isLetter` is the opposite of the value in T2 but `last == s` and `last == r` remain the same. We find this combination in T6.

We have found another pair of tests, T2 and T6, where `isLetter` is the only parameter that is different and the outcome (decision) also changes, which we also add to our list of test cases.

- We repeat the process for T3 (`isLetter` is true, `last == s` is false, `last == r` is true) and find that the `isLetter` parameter in T7 (`isLetter` is false, `last == s` is false, `last == r` is true) is the opposite of the value in T3 and changes the outcome (decision).
- The pair for T4 (`isLetter` is true, `last == s` is false, `last == r` is false) is T8 (`isLetter` is false, `last == s` is false, `last == r` is false). The outcome of both tests is the same (decision is false), which means the pair {T4, T8} does *not* show how `isLetter` can independently affect the overall outcome.

We do not find another new or suitable pair when repeating the process for T5, T6, T7, and T8, so we move on from the `isLetter` parameter to the `last == s` parameter. We repeat the same process, but now we search for the opposite value of parameter `last == s`, while `isLetter` and `last == r` stay the same:

- For T1 (`isLetter` is true, `last == s` is true, `last == r` is true), we search for a test where `isLetter` is true, `last == s` is false, `last == r` is true). This appears to be the case in T3. However, the outcome is the same for both test cases. Therefore, {T1, T3} does *not* show how the `last == s` parameter independently affects the outcome.
- After repeating all the steps for the other tests, we find that only {T2, T4} have different values for the `last == s` parameter where the outcome also changes.

Finally, we move to the `last == r` parameter. As with the `last == s` parameter, one pair of combinations works: {T3, T4}. I highly recommend carrying out the entire process yourself to get a feel for how it works.

We now have all the pairs for each parameter:

- `isLetter`: {1, 5}, {2, 6}, {3, 7}
- `last == s`: {2, 4}
- `last == r`: {3, 4}

Having a single independence pair per variable (`isLetter`, `last == s`, and `last == r`) is enough. We want to minimize the total number of tests, and we know we can achieve this with $N + 1$ tests. We do not have any choices with conditions `last == s` and `last == r`, as we found only one pair of tests for each parameter. This means we need tests T2, T3, and T4. Finally, we need to find the appropriate pair of tests for `isLetter`. Note that any of the test pairs (T1-T5, T2-T6, or T3-T7) would work. However, we want to reduce the total number of tests in the test suite (and again, we know we only need four in this case).

If we were to pick T1 or T5, we would have to include the other as well, as they are opposites. Therefore, they are unnecessarily increasing the number of tests. To ensure that our test suite contains at most four test cases, we can add either T6 or T7, as their opposites (T2 and T3) are already included in our test cases. I picked T6 randomly. (You can have more than one set of tests that achieves 100% MC/DC, and all solutions are equally acceptable.)

Therefore, the tests we need for 100% MC/DC coverage are {T2, T3, T4, T6}. These are the only four tests we need—certainly cheaper than the eight tests we would need for path coverage. Now that we know which tests we need to implement, we can automate them.

NOTE I have a video on YouTube that explains MC/DC visually: www.youtube.com/watch?v=HzmnCVaICQ4.

3.5 Handling loops and similar constructs

You may wonder what to do in the case of loops, such as `for` and `while`. The code block inside the loop may be executed different numbers of times, making testing more complicated.

Think of a `while(true)` loop, which can be non-terminating. To be rigorous, we would have to test the program with the loop block executed one time, two times, three times, and so on. Or imagine a `for(i = 0; i < 10; i++)` loop with a `break` inside the body. We would have to test what happened if the loop body executed up to 10 times. How can we handle a long-lasting loop (that runs for many iterations) or an unbounded loop (that is executed an unknown number of times)?

Given that exhaustive testing is impossible, testers often rely on the *loop boundary adequacy criterion* to decide when to stop testing a loop. A test suite satisfies this criterion if and only if for every loop

- There is a test case that exercises the loop zero times.
- There is a test case that exercises the loop once.
- There is a test case that exercises the loop multiple times.

Pragmatically speaking, my experience shows that the main challenge comes when devising the test case for the loop being executed multiple times. Should the test case force the loop to iterate 2, 5, or 10 times? This decision requires a good understanding of the program and its requirement. With optimal understanding of the specs, you should be able to devise good tests for the loop. Do not be afraid to create two or more tests for the “multiple times” case. Do whatever you need to do to ensure that the loop works as expected.

3.6 Criteria subsumption, and choosing a criterion

You may have noticed that some of the criteria we have discussed are more rigorous than others. For example, a single test is enough to achieve 100% line coverage, but two tests are needed for 100% branch coverage. Some strategies *subsume* other strategies.

Formally, a strategy X subsumes strategy Y if all elements that Y exercises are also exercised by X. Figure 3.7 illustrates the relationships among the coverage criteria.

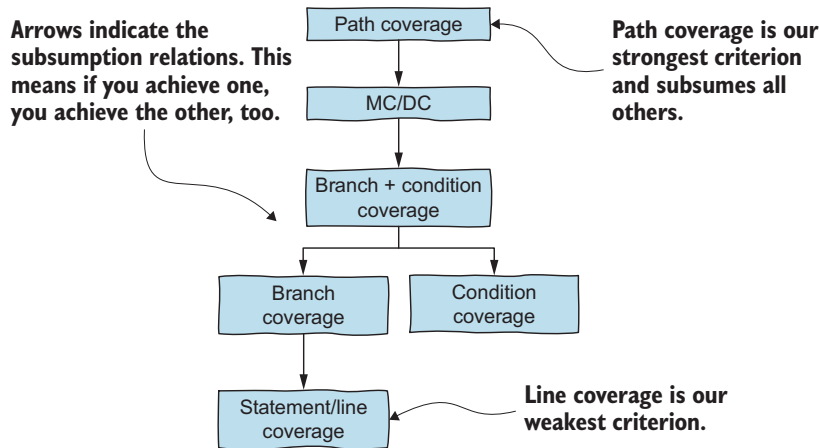


Figure 3.7 The different coverage criteria and their subsumption relations

Branch coverage subsumes line coverage, which means 100% branch coverage always implies 100% line coverage. However, 100% line coverage does not imply 100% branch coverage. Moreover, 100% condition + branch coverage always implies 100% branch coverage and 100% line coverage. Following this train of thought, we see that path coverage subsumes all other criteria. This is logical as path coverage covers all possible paths of the program. Next, we see that MC/DC is stronger than condition + branch coverage, as MC/DC ensures the independence of each condition. And condition + branch coverage subsumes both branch and condition coverage independently. Finally, all other criteria, except basic condition coverage, subsume line coverage, which is the weakest criterion in the figure.

You now understand the trade-offs of choosing one criterion over another. A weaker criterion may be cheaper and faster to achieve but leave many parts of the code uncovered. On the other hand, a stronger criterion may cover the code more rigorously at a higher cost. It is up to you, the developer, to decide which criterion to use.

NOTE Basic condition coverage does not necessarily subsume line coverage, for the same reason we always use condition + branch coverage together. We can achieve 100% basic condition coverage in a simple `if (A || B)` by having two tests, `T1={true, false}` and `T2={false, true}`. But both tests make the decision block true, so the false branch and its lines are not exercised.

3.7 Specification-based and structural testing: A running example

Let's try specification-based testing and structural testing together on a real-world example: the `leftPad()` function from Apache Commons Lang (<http://mng.bz/zQ2g>):

Left-pad a string with a specified string. Pad to a size of `size`.

- `str`—The string to pad out; may be null.
- `size`—The size to pad to.
- `padStr`—The string to pad with. Null or empty is treated as a single space.

The method returns a left-padded string, the original string if no padding is necessary, or null if a null string is input.

For example, if we give "abc" as the string input, a dash "-" as the pad string, and 5 as the size, the program will output "--abc".

A developer on your team comes up with the implementation in listing 3.5. For now, suppose you are testing code written by others, so you need to build an understanding of the code before you can test it properly. Specification-based testing and structural testing are applied the same way, regardless of whether you wrote the code. In later chapters, we discuss test-driven development and how you can use tests to guide you through implementation.

Listing 3.5 `leftPad` implementation from the Apache Commons

```
public static String leftPad(final String str, final int size,
    String padStr) {

    if (str == null) {
        return null;
    }

    if (padStr == null || padStr.isEmpty()) {
        padStr = SPACE;
    }

    final int padLen = padStr.length();
    final int strLen = str.length();
    final int pads = size - strLen;

    if (pads <= 0) {
        // returns original String when possible
        return str;
    }

    if (pads == padLen) {
        return padStr.concat(str);
    }
    else if (pads < padLen) {
        return padStr.substring(0, pads).concat(str);
    }
}
```

← If the string to pad is null, we return null right away.

← If the pad string is null or empty, we make it a space.

← There is no need to pad this string.

← If the number of characters to pad matches the size of the pad string, we concatenate it.

← If we cannot fit the entire pad string, we add only the part that fits.

```

    } else {
        final char[] padding = new char[pads];
        final char[] padChars = padStr.toCharArray();

        for (int i = 0; i < pads; i++) {
            padding[i] = padChars[i % padLen];
        }

        return new String(padding).concat(str);
    }
}

```

← We have to add the pad string more than once. We go character by character until the string is fully padded.

Now it is time for some systematic testing. As we know, the first step is to apply specification-based testing. Let's follow the process discussed in chapter 2 (I suggest you try to do it yourself and compare your solution to mine):

- 1 We read the requirements. We understand that the program adds a given character/string to the beginning (left) of the string, up to a specific size. The program has three input parameters: `str`, representing the original string to be padded; `size`, representing the desired size of the returned string; and `padStr`, representing the string used to pad. The program returns a `String`. The program has specific behavior if any of the inputs is null. (If we had implemented the feature ourselves, we would probably skip this step, as we would already have a complete understanding of the requirements.)
- 2 Based on all the observations in step 1, we derive the following list of partitions:
 - `str` parameter
 - Null
 - Empty string
 - Non-empty string
 - `size` parameter
 - Negative number
 - Positive number
 - `padStr` parameter
 - Null
 - Empty
 - Non-empty
 - `str`, `size` parameters
 - `size < len(str)`
 - `size > len(str)`
- 3 There are several boundaries:
 - `size` being precisely 0
 - `str` having length 1
 - `padStr` having length 1
 - `size` being precisely the length of `str`

- 4 We can devise single tests for exceptional cases such as null, empty, and negative size. We also have a boundary related to `padStr`: we can exercise `padStr` with a single character only once and have all other tests use a pad with a single character (otherwise, the number of combinations would be too large). We obtain the following tests:

- T1: `str` is null.
- T2: `str` is empty.
- T3: negative size.
- T4: `padStr` is null.
- T5: `padStr` is empty.
- T6: `padStr` has a single character.
- T7: `size` is equal to the length of `str`.
- T8: `size` is equal to 0.
- T9: `size` is smaller than the length of `str`.

Now we automate the tests. I used a parameterized test, but it is fine if you prefer nine traditional JUnit tests.

Listing 3.6 Tests for `LeftPad` after specification-based testing

```
public class LeftPadTest {

    @ParameterizedTest
    @MethodSource("generator")
    void test(String originalStr, int size, String padString,
              String expectedStr) {
        assertThat(leftPad(originalStr, size, padString))
            .isEqualTo(expectedStr);
    }

    static Stream<Arguments> generator() {
        return Stream.of(
            of(null, 10, "-", null),
            of("", 5, "-", "-----"),
            of("abc", -1, "-", "abc"),
            of("abc", 5, null, " abc"),
            of("abc", 5, "", " abc"),
            of("abc", 5, "-", "--abc"),
            of("abc", 3, "-", "abc"),
            of("abc", 0, "-", "abc"),
            of("abc", 2, "-", "abc")
        );
    }
}
```

← The parameterized test, similar to the ones we have written before

← The nine tests we created are provided by the method source.

T1 →

T2 →

T4 →

T6 →

T8 →

← T3

← T5

← T7

← T9

It is time to augment the test suite through structural testing. Let's use a code coverage tool to tell us what we have already covered (see figure 3.8). The report shows that we are missing some branches: the `if (pads == padLen)` and `else if (pads < padLen)` expressions.



Figure 3.8 Code coverage achieved by the specification-based tests for the `leftPad` method. The two return lines near the arrow are not covered; the `if` and `else if`, also near the arrow, are only partially covered. The remaining lines are fully covered.

This is useful information. Why didn't we cover these lines? What did we miss? As a developer, you should triangulate what you see in the source with the specification and your mental model of the program. In this case, we conclude that we did not exercise `padStr` being smaller, greater, or equal to the remaining space in `str`. What a tricky boundary! This is why structural testing is essential: it helps identify partitions and boundaries we may have missed.

With that information in mind, we derive three more test cases:

- T10: the length of `padStr` is equal to the remaining spaces in `str`.
- T11: the length of `padStr` is greater than the remaining spaces in `str`.
- T12: the length of `padStr` is smaller than the remaining spaces in `str` (this test may be similar to T6).

We add these three extra test cases to our parameterized test, as shown in listing 3.7. When we run the coverage tool again, we get a report similar to the one in figure 3.9. We now cover all the branches.

Listing 3.7 Three new test cases for `leftPad`

```

static Stream<Arguments> generator() {
    return Stream.of(
        // ... others here
    );
}

```

```

of("abc", 5, "--", "--abc"), // T10
of("abc", 5, "---", "--abc"), // T11
of("abc", 5, "-", "--abc") // T12
);
}

```

All lines are green.
Everything is covered!

```

public static String leftPad(final String str, final int size, String padStr) {
    if (str == null) {
        return null;
    }
    if (isEmpty(padStr)) {
        padStr = SPACE;
    }
    final int padLen = padStr.length();
    final int strLen = str.length();
    final int pads = size - strLen;
    if (pads <= 0) {
        return str; // returns original String when possible
    }

    if (pads == padLen) {
        return padStr.concat(str);
    } else if (pads < padLen) {
        return padStr.substring(0, pads).concat(str);
    } else {
        final char[] padding = new char[pads];
        final char[] padChars = padStr.toCharArray();
        for (int i = 0; i < pads; i++) {
            padding[i] = padChars[i % padLen];
        }
        return new String(padding).concat(str);
    }
}

```

Figure 3.9 Code coverage of the `leftPad` method after specification-based and structural tests. We now achieve 100% branch coverage.

NOTE Interestingly, if you look at the entire class, JaCoCo does not give 100% coverage, but only 96%. The report highlights the first line of the file: the declaration of the class, `public class LeftPadUtils {`. The `leftPad` method is static, so none of our tests instantiate this class. Given that we know the context, we can ignore the fact that this line is not covered. This is a good example of why only looking at the numbers makes no sense. We discuss this further, later in the chapter.

With all the branches covered, we now look for other interesting cases to test. The implementation contains interesting decisions that we may decide to test. In particular, we observe an `if (pads <= 0)` block with the code comment “returns original String when possible”. As a tester, you may decide to test this specific behavior: “If the string is not padded, the program should return the same String instance.” That can be written as a JUnit test as follows.

Listing 3.8 Another extra test for `leftPad`

```
@Test
void sameInstance() {
    String str = "sometext";
    assertThat(leftPad(str, 5, "-")).isSameAs(str);
}
```

We are now much more confident that our test suite covers all the critical behavior of the program. Structural testing and code coverage helped us identify parts of the code that we did not test (or partitions we missed) during our specification-based testing—and that is what structural testing is all about.

3.8 *Boundary testing and structural testing*

The most challenging part of specification-based testing is identifying boundaries. They are tricky to find, given the way we write specifications. Luckily, they are much easier to find in source code, given how precise code has to be. All the boundary testing ideas we discussed in the previous chapter apply here.

The idea of identifying and testing on and off points fits nicely in structural testing. For example, we can analyze the `if` statements in the `leftPad` program:

- `if (pads <= 0)`—The on point is 0 and evaluates the expression to `true`. The off point is the nearest point to the on point that makes the expression evaluate to `false`. In this case, given that `pads` is an integer, the nearest point is 1.
- `if (pads == padLen)`—The on point is `padLen`. Given the equality and that `padLen` is an integer, we have two off points: one that happens when `pads == padLen - 1` and another that happens when `pads = padLen + 1`.
- `if (pads < padLen)`—The on point is again `padLen`. The on point evaluates the expression to `false`. The off point is, therefore, `pads == padLen - 1`.

As a tester, you may want to use this information to see whether you can augment your test suite.

We discussed the loop boundary criterion earlier, which helps us try different possible boundaries. If a loop has a less conventional, more complicated expression, consider applying on and off analysis there as well.

3.9 *Structural testing alone often is not enough*

If code is the source of all truth, why can't we just do structural testing? This is a very interesting question. Test suites derived only with structural testing can be reasonably effective, but they may not be strong enough. Let's look at an example (see the "counting clumps" problem, inspired by a CodingBat assignment: <https://codingbat.com/prob/p193817>):

The program should count the number of clumps in an array. A clump is a sequence of the same element with a length of at least 2.

- **nums**—The array for which to count the clumps. The array must be non-null and `length > 0`; the program returns 0 if any pre-condition is violated.

The program returns the number of clumps in the array.

The following listing shows an implementation.

Listing 3.9 Implementing the code clumps requirement

```
public static int countClumps(int[] nums) {
    if (nums == null || nums.length == 0) {
        return 0;
    }
    int count = 0;
    int prev = nums[0];
    boolean inClump = false;
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] == prev && !inClump) {
            inClump = true;
            count += 1;
        }
        if (nums[i] != prev) {
            prev = nums[i];
            inClump = false;
        }
    }
    return count;
}
```

← If null or empty (pre-condition), return 0 right away.

← If the current number is the same as the previous number, we have identified a clump.

← If the current number differs from the previous one, we are not in a clump.

Suppose we decide not to look at the requirements. We want to achieve, say, 100% branch coverage. Three tests are enough to do that (T1–T3). Maybe we also want to do some extra boundary testing and decide to exercise the loop, iterating a single time (T4):

- T1: an empty array
- T2: a null array
- T3: an array with a single clump of three elements in the middle (for example, [1, 2, 2, 2, 1])
- T4: an array with a single element

To check that for yourself, write down these three tests as (JUnit) automated test cases and run your favorite code coverage tool as in the following.

Listing 3.10 100% branch coverage for the clump-counting problem

```
@ParameterizedTest
@MethodSource("generator")
void testClumps(int[] nums, int expectedNoOfClumps) {
    assertEquals(Clumps.countClumps(nums), expectedNoOfClumps);
}
```

```

        .isEqualTo(expectedNoOfClumps);
    }

    static Stream<Arguments> generator() {
        return Stream.of(
            of(new int[] {}, 0), // empty
            of(null, 0), // null
            of(new int[] {1,2,2,2,1}, 1), // one clump
            of(new int[] {1}, 0) // one element
        );
    }
}

```

← The four test cases we defined

This test suite is reasonable and exercises the main behavior of the program, but note how weak it is. It achieves 100% branch coverage, but it misses many interesting test cases. Even without performing systematic specification testing, in a program that counts clumps, it is natural to try the program with multiple clumps instead of just one. We could try it with the last clump happening at the last item of the array or with an array that has a clump starting in the first position. Such specific cases cannot be captured by pure structural testing guided mainly by coverage. This is yet another reason not to rely blindly on coverage. Structural testing shows its value when combined with knowledge of the specification.

3.10 *Structural testing in the real world*

Now that you have a clear picture of structural testing, the coverage criteria you can use for guidance, and how to use structural testing in combination with specification-based testing, let me discuss a few interesting points.

3.10.1 *Why do some people hate code coverage?*

I find it interesting that some people rage against code coverage. A prevalent opinion is, “If I write a test case with no assertions, I achieve 100% coverage, but I am not testing anything!” This is true. If your tests have no assertions, they do not test anything, but the production code is exercised. However, I consider that a flawed argument. It assumes the very worst (unrealistic) scenario possible. If you are writing test suites with no assertions, you have bigger problems to take care of before you can enjoy the benefits of structural testing.

Between the lines, people use such an argument to explain that you should not look at the coverage number blindly, because it can mislead you. That I fully agree with. Here, the misconception is how people see code coverage. If code coverage is only a number you should achieve, you may end up writing less useful test cases and gaming the metric (something that Bouwers, Visser, and Van Deursen have argued in 2012).

I hope this chapter has clarified how structural testing and code coverage should be used: to augment specification-based testing, quickly identify parts of the code that are not currently exercised by the test suite, and identify partitions you missed when

doing specification-based testing. Achieving a high coverage number may be a consequence of you doing that, but the purpose is different. If you leave a line uncovered, it is because you thought about it and decided not to cover it.

EMPIRICAL EVIDENCE IN FAVOR OF CODE COVERAGE

Understanding whether structural coverage helps and whether high coverage numbers lead to better-tested software has been the goal of many empirical software engineering researchers. Interestingly, while researchers have not yet found a magical coverage number that we should aim for, some evidence points toward the benefits of structural testing. I quote four of these studies:

- *Hutchins et al. (1994)*—“Within the limited domain of our experiments, test sets achieving coverage levels over 90% usually showed significantly better fault detection than randomly chosen test sets of the same size. In addition, significant improvements in the effectiveness of coverage-based tests usually occurred as coverage increased from 90% to 100%. However, the results also indicate that 100% code coverage alone is not a reliable indicator of the effectiveness of a test set.”
- *Namin and Andrews (2009)*—“Our experiments indicate that coverage is sometimes correlated with effectiveness when test suite size is controlled for, and that using both size and coverage yields a more accurate prediction of effectiveness than test suite size alone. This, in turn, suggests that both size and coverage are important to test suite effectiveness.”
- *Inozemtseva and Holmes (2014)*—“We found that there is a low to moderate correlation between coverage and effectiveness when the number of test cases in the suite is controlled for. In addition, we found that stronger forms of coverage do not provide greater insight into the effectiveness of the suite. Our results suggest that coverage, while useful for identifying under-tested parts of a program, should not be used as a quality target because it is not a good indicator of test suite effectiveness.”
- *Gopinath et al. (2020)*—“This paper finds a correlation between lightweight, widely available coverage criteria (statement, block, branch, and path coverage) and mutation kills for hundreds of Java programs (...). For both original and generated suites, statement coverage is the best predictor for mutation kills, and in fact does a relatively good job of predicting suite quality.”

Although developing sound experiments to show whether coverage helps is difficult, and we are not quite there yet (see Chen et al.’s 2020 paper for a good statistical explanation of why it is hard), the current results make sense to me. Even with the small code examples we have been exploring, we can see a relationship between covering all the partitions via specification-based testing and covering the entire source code. The opposite is also true: if you cover a significant part of the source code, you also cover most of the partitions. Therefore, high coverage implies more partitions being tested.

The empirical results also show that coverage alone is not always a strong indicator of how good a test suite is. We also noticed that in the test cases we derived for the `CountWords` problem at the beginning of this chapter. We purposefully did bad specification-based testing and then augmented the test suite with structural testing. We ended up with three test cases that achieve 100% condition + branch coverage. But is the test suite strong enough? I don't think so. I can think of many extra test cases that would touch the same lines and branches again but would nonetheless make the test suite much more effective against possible bugs.

On the other hand, although 100% coverage does not necessarily mean the system is properly tested, having very low coverage *does* mean your system is *not* properly tested. Having a system with, say, 10% coverage means there is much to be done as far as testing.

I suggest reading Google's code coverage best practices (Arguelles, Ivankovic, and Bender, 2020). Their perceptions are in line with everything we have discussed here.

3.10.2 What does it mean to achieve 100% coverage?

I have purposefully skipped talking much about achieving 100% line coverage or branch coverage or other coverage. I do not believe that achieving a number should be the goal. Nevertheless, given how prevalent those numbers are in practice, it is important to understand them. First, let's talk about the metrics themselves.

NOTE Formulas vary among the tools on the market. Check your tool's manual to better understand the precise numbers you get.

If the entire test suite covers all the lines in the program (or in the class or method under test), that suite achieves *100% line coverage*. A simple formula to calculate the line coverage of a given program or method is to divide the number of lines covered by the total number of lines:

$$\text{line coverage} = \frac{\text{lines covered}}{\text{total number of lines}} \times 100\%$$

You can calculate this number at the method level, class level, package level, system level, or whatever level you are interested in.

Similar to line coverage, a formula to calculate the achieved *branch coverage* of a program or method is the number of branches covered divided by the total number of branches:

$$\text{branch coverage} = \frac{\text{branches covered}}{\text{total number of branches}} \times 100\%$$

In a simple program such as `if (x) { do A } else { do B }`, the total number of branches is two (the single `if` statement branches the program in two ways). Therefore, if one

test in your test suite covers, say $x = \text{true}$, your test suite achieves $1/2 \times 100\% = 50\%$ branch coverage. Note that due to criteria subsumption, which we discussed earlier, if you cover all the branches of the program, you also cover all the lines.

Finally, a formula to calculate the *condition + branch coverage* of a given program or method is the sum of all branches and conditions covered, divided by the total number of branches and conditions:

$$\text{c+b coverage} = \frac{\text{branches covered} + \text{conditions covered}}{\text{number of branches} + \text{number of conditions}} \times 100\%$$

In a simple program such as `if(x || y) { do A } else { do B }`, the total number of branches is two (the single `if` statement branches the program in two ways) and the total number of conditions is four (two conditions for `x` and two conditions for `y`). Therefore, if you have two tests in your test suite—T1: (true, true) and T2: (false, true)—the test suite achieves $(1 + 3)/(2 + 4) \times 100\% = 66.6\%$ condition + branch coverage. The test suite covers only one branch of the program (the true branch, as both T1 and T2 make the `if` expression evaluate to true), and three of the four conditions (`x` is exercised as true and false, but `y` is only exercised as true).

Figure 3.10 shows a simple illustration of line coverage, branch coverage, and condition + branch coverage. When someone says, “My test suite achieves 80% condition + branch coverage,” you now understand that 80% of the branches and conditions are covered by at least one test case. And when someone says, “My test suite achieves 100% line coverage,” you know that 100% of the lines are covered by at least one test case.

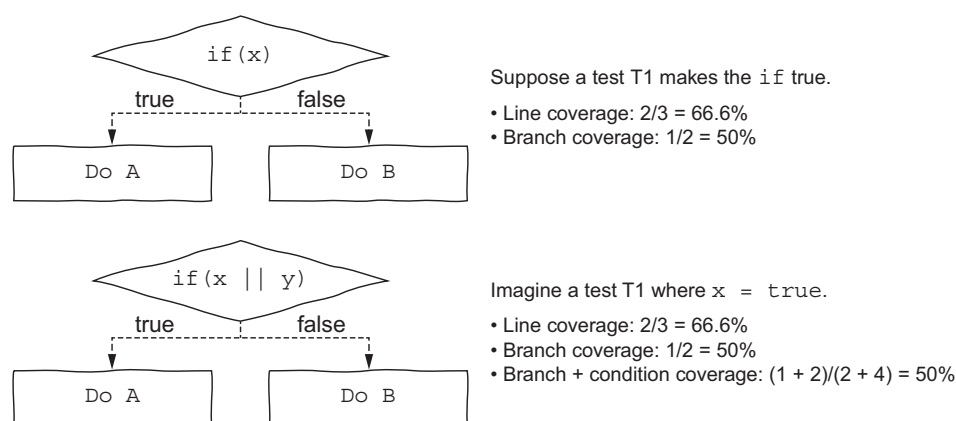


Figure 3.10 Two control-flow graphs of simple programs and how the different coverage criteria are calculated

3.10.3 What coverage criterion to use

This is a popular question among practitioners and researchers. If we settle for a less-rigorous criterion, such as line coverage instead of branch coverage, we might miss something. Plus this question brings the focus back to the metric, which we do not want.

Which criterion to use depends on the context: what you are testing at that moment and how rigorous you want the testing to be. Structural testing is meant to complement specification-based testing. When you dive into the source code and look for uncovered parts, you may decide to use branch coverage for a specific `if` expression but MC/DC for another `if` expression. This makes the approach less systematic (and, therefore, more prone to errors and different developers using different criteria), but it is the most pragmatic approach I know. You may want to perform some risk assessment to determine how important it is to be thorough.

My rule of thumb is branch coverage: I always try to at least reach all the branches of the program. Whenever I see a more complicated expression, I evaluate the need for condition + branch coverage. If I see an even more complex expression, I consider MC/DC.

3.10.4 MC/DC when expressions are too complex and cannot be simplified

MC/DC is increasingly valuable as expressions become more complicated. Listing 3.11 shows an example of a complex expression that I extracted from Chilenski's 2001 paper. It is an anonymized version of a condition found in a level A flight simulation program and contains an impressive 76 conditions. Achieving path coverage in such a complex expression is impossible ($2^{76} = 7.5 \times 10^{22}$ test cases), so smart approaches such as MC/DC come in handy.

Listing 3.11 Complex expression from flight simulation software

```
Bv or (Ev != El) or Bv2 or Bv3 or Bv4 or Bv5 or Bv6 or Bv7 or Bv8 or Bv9 or
Bv10 or Bv11 or Bv12 or Bv13 or Bv14 or Bv15 or Bv16 or Bv17 or Bv18 or
Bv19 or Bv20 or Bv21 or Bv22 or Bv23 or Bv24 or Bv25 or Bv26 or Bv27 or
Bv28 or Bv29 or Bv30 or Bv31 or Bv32 or Bv33 or Bv34 or Bv35 or Bv36 or
Bv37 or Bv38 or Bv39 or Bv40 or Bv41 or Bv42 or Bv43 or Bv44 or Bv45 or
Bv46 or Bv47 or Bv48 or Bv49 or Bv50 or Bv51 or (Ev2 = El2) or
((Ev3 = El2) and (Sav != Sac)) or Bv52 or Bv53 or Bv54 or Bv55 or Bv56
or Bv57 or Bv58 or Bv59 or Bv60 or Bv61 or Bv62 or Bv63 or Bv64 or Bv65
or Ev4 != El3 or Ev5 = El4 or Ev6 = El4 or Ev7 = El4 or Ev8 = El4 or
Ev9 = El4 or Ev10 = El4
```

Pragmatically speaking, testing such a complex expression, with or without MC/DC, is a challenge, and you should avoid doing so when possible. Sometimes you can break an expression into smaller bits that you can then test. But in cases where breaking complex expressions is not possible, MC/DC shines.

MC/DC in SQLite

A nice story of the benefits of MC/DC was told by Richard Hipp, the creator and primary developer of SQLite, the most popular embedded database. In the Corecursive #066 podcast, Richard says, “I had this idea, I’m going to write tests to bring SQLite up to the quality of 100% MC/DC, and that took a year of 60-hour weeks. That was hard, hard work. I was putting in 12-hour days every single day. I was getting so tired of this because with this sort of thing, it’s the old joke of, you get 95% of the functionality with the first 95% of your budget, and the last 5% on the second 95% of your budget. It’s kind of the same thing. It’s pretty easy to get up to 90 or 95% test coverage. Getting that last 5% is really, really hard, and it took about a year for me to get there, but once we got to that point, we stopped getting bug reports from Android.” What a powerful success story of MC/DC.

For completeness, here are some final remarks about MC/DC. First, in the example in section 3.1, we apply *unique-cause MC/DC criteria*: we identify an independence pair (T1, T2) where only a single condition and the final outcome change between T1 and T2. That may not be possible in all cases. For example, consider $(A \ \&\& \ B) \ || \ (A \ \&\& \ C)$. Ideally, we would demonstrate the independence of the first A, B, the second A, and C. But it is impossible to change the first A and not change the second A. Thus, we cannot demonstrate the independence of each A in the expression. In such cases, we allow A to vary, but we fix all other variables (this is called *masked MC/DC*).

Second, note that it may not be possible to achieve MC/DC in some expressions, such as $(A \ \text{and} \ B) \ \text{or} \ (A \ \text{and} \ \text{not} \ B)$. While the independence pairs (TT, FT) would show the independence of A, there are no pairs that show the independence of B. In such cases, revisit the expression, as it may have been poorly designed. In this example, the expression could be reformulated to simply A.

Finally, mathematically speaking, $N + 1$ is the theoretical lower bound for the number of tests you may need when applying MC/DC. In other words, you may need more than $N + 1$ test cases to achieve MC/DC in some expressions. However, the empirical study by Chilenski (2001) shows that the majority of expressions in practice require $N + 1$ tests. This has been my observation, too: $N + 1$ is most of the times the number of required test cases.

3.10.5 Other coverage criteria

Throughout this chapter, we have used the program’s control flow as a way to derive different tests. Another way of approaching structural testing is to look at the *data flow*: examining how the data flows to different parts of the program.

For example, imagine that a variable is defined, then modified one, two, or three times in other parts of the program, and then used again later. You may want to ensure that you exercise all the possible ways this variable is touched. Trying to summarize data-flow coverage in one sentence is unfair, and a lot of energy has been spent coming up with criteria, but this should give you some intuition about it.

I do not discuss data-flow coverage in this book, but I suggest you read more about it. Pezzè and Young (2008) give a nice explanation.

3.10.6 What should not be covered?

We have talked a lot about what to test and cover. Let's quickly discuss what *not* to cover. Achieving 100% coverage may be impossible or not even desirable. For example, the code snippet in listing 3.12 returns the full path of a specific directory. The code may throw a `URISyntaxException`, which we catch and wrap around a `RuntimeException`. (For the Java experts, we are converting a checked exception to an unchecked exception.)

Listing 3.12 A method that does not deserve full coverage

```
public static String resourceFolder(String path) {
    try {
        return Paths.get(ResourceUtils.class
            .getResource("/").toURI()).toString() + path;
    } catch (URISyntaxException e) {
        throw new RuntimeException(e);
    }
}
```

To achieve 100% line coverage, we would need to exercise the catch block. For that to happen, we would have to somehow force the `toURI` method to throw the exception. We could use mocks (discussed later in this book), but I cannot see any advantage in doing that. It is more important to test what would happen to the rest of the system if `resourceFolder` threw a `RuntimeException`. That is much easier to do, as we have more control over the `resourceFolder` method than the Java `toURI()` method. Therefore, this piece of code it is not worth covering and shows why blindly aiming for 100% coverage makes no sense.

In Java, in particular, I tend not to write dedicated tests for `equals` and `hashCode` methods or straightforward getters and setters. These are tested implicitly by the tests that exercise the other methods that use them.

To close this discussion, I want to reinforce that, for me, *all code should be covered until proven otherwise*. I start from the idea that I should have 100% coverage. Then, if I see that a piece of code does not need to be covered, I make an exception. But be careful—experience shows that bugs tend to appear in areas you do not cover well.

3.11 Mutation testing

All the coverage criteria discussed in this chapter consider how much of the production code is exercised by a test. What they all miss is whether the assertions that these tests make are good and strong enough to capture bugs. If we introduce a bug in the code, even in a line covered by a test, will the test break?

As mentioned earlier, coverage alone is not enough to determine whether a test suite is good. We have been thinking about how far our test suite goes to evaluate the

strength of our test suite. Now let's think of the test suite's *fault detection capability*. How many bugs can it reveal?

This is the idea behind *mutation testing*. In a nutshell, we purposefully insert a bug in the existing code and check whether the test suite breaks. If it does, that's a point for the test suite. If it does not (all tests are green even with the bug in the code), we have found something to improve in our test suite. We then repeat the process: we create another buggy version of the problem by changing something else in the code, and we check whether the test suite captures that bug.

These buggy versions are *mutants* of the original, supposedly correct, version of the program. If the test suite breaks when executed against a mutant, we say that the test suite *kills* that mutant. If it does not break, we say that the mutant *survives*. A test suite achieves 100% mutation coverage if it kills all possible mutants.

Mutation testing makes two interesting assumptions. First, the *competent programmer hypothesis* assumes that the program is written by a competent programmer and that the implemented version is either correct or differs from the correct program by a combination of simple errors. Second, the *coupling effect* says that a complex bug is caused by a combination of many small bugs. Therefore, if your test suite can catch simple bugs, it will also catch the more complex ones.

Pitest is the most popular open source tool for mutation testing in Java (<https://pitest.org/quickstart/mutators>). Here are a few examples of mutators from its manual:

- *Conditionals boundary*—Relational operators such as `<` and `<=` are replaced by other relational operators.
- *Increment*—It replaces `i++` with `i--` and vice versa.
- *Invert negatives*—It negates variables: for example, `i` becomes `-i`.
- *Math operators*—It replaces mathematical operators: for example, a plus becomes a minus.
- *True returns*—It replaces entire boolean variables with `true`.
- *Remove conditionals*—It replaces entire `if` statements with a simple `if (true) {...}`.

Running Pitest is simple, as it comes with plugins for Maven and Gradle. For example, I ran it against the `LeftPad` implementation and tests we wrote earlier; figure 3.11 shows the resulting report. As in a code coverage report, a line's background color indicates whether all the mutants were killed by the test suite.

The next step is to evaluate the surviving mutants. It is very important to analyze each surviving mutant, as some may not be useful.

Remember that mutation testing tools do not know your code—they simply mutate it. This sometimes means they create mutants that are not useful. For example, in the line that contains `int pads = size - strLen`, Pitest mutated the `size` variable to `size++`. Our test suite does not catch this bug, but this is not a useful mutant: the `size` variable is not used after this line, so incrementing it has no effect on the program.

You should view mutation testing in the same way as coverage tools: it can augment the test suite engineered based on the program's requirements.

```

22     public static String leftPad(final String str, final int size, String padStr) {
23 4     if (str == null) {
24 2     return null;
25     }
26 9     if (isEmpty(padStr)) {
27     padStr = SPACE;
28     }
29 1    final int padLen = padStr.length();
30 1    final int strLen = str.length();
31 17   final int pads = size - strLen;
32 14   if (pads <= 0) {
33 2    return str; // returns original String when possible
34     }
35
36 18   if (pads == padLen) {
37 5    return padStr.concat(str);
38 19   } else if (pads < padLen) {
39 17   return padStr.substring(0, pads).concat(str);
40     } else {
41 5    final char[] padding = new char[pads];
42 1    final char[] padChars = padStr.toCharArray();
43 25   for (int i = 0; i < pads; i++) {
44 22   padding[i] = padChars[i % padLen];
45     }
46 6    return new String(padding).concat(str);
47     }
48     }
49
50 }

```

Figure 3.11 Part of a report generated by Pitest. Lines 26, 31, 32, 36, 38, 39, 43, and 44 have surviving mutants.

Mutation testing faces various challenges in practice, including the cost. To use mutation testing, we must generate many mutants and execute the whole test suite with each one. This makes mutation testing quite expensive. Considerable research is dedicated to lowering the cost of mutation testing, such as reducing the number of mutants to try, detecting equivalent mutants (mutants that are identical to the original program in terms of behavior), and reducing the number of test cases or test case executions (see the work of Ferrari, Pizzoleto, and Offutt, 2018). As a community, we are taking steps toward a solution, but we are not there yet.

Despite the cost, mutation testing is highly beneficial. In a very recent paper by Parsai and Demeyer (2020), the authors demonstrate that mutation coverage reveals additional weaknesses in the test suite compared to branch coverage and that it can do so with an acceptable performance overhead during project build. Even large companies like Google are investing in mutation testing in their systems, as reported by Petrović and Ivanković (2018).

Researchers are also exploring mutation testing in areas other than Java backend code. Yandrapally and Mesbah (2021) propose mutations for the Document Object Model (DOM) in HTML pages to assess whether web tests (which we discuss in

chapter 9) are strong enough. In addition, Tuya and colleagues (2006) proposed the use of mutation in SQL queries.

I suggest that you try to apply mutation testing, especially in more sensitive parts of your system. While running mutation testing for the entire system can be expensive, running it for a smaller set of classes is feasible and may give you valuable insights about what else to test.

Exercises

3.1 Consider the following piece of code, which plays a game of Blackjack:

```
01. public int play(int left, int right) {  
02.     int ln = left;  
03.     int rn = right;  
04.     if (ln > 21)  
05.         ln = 0;  
06.     if (rn > 21)  
07.         rn = 0;  
08.     if (ln > rn)  
09.         return ln;  
10.     else  
11.         return rn;  
12. }
```

What is the line coverage of a test where `left=22` and `right=21`? In the calculation, disregard the lines with the function signature and the last curly bracket (lines 1 and 12).

- A** 60%
- B** 80%
- C** 70%
- D** 100%

3.2 Consider the following `remove` method:

```
public boolean remove(Object o) {  
    if (o == null) {  
        for (Node<E> x = first; x != null; x = x.next) {  
            if (x.item == null) {  
                unlink(x);  
                return true;  
            }  
        }  
    } else {  
        for (Node<E> x = first; x != null; x = x.next) {  
            if (o.equals(x.item)) {  
                unlink(x);  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

This is the implementation of the Java Platform, Standard Edition 8 Development Kit (JDK 8) `LinkedList` remove method.

Create a test suite (a set of tests) that achieves 100% line coverage. Use as few tests as possible. Feel free to write them as JUnit tests or as a set of inputs and expected outputs.

- 3.3** Following is Java's implementation of the `LinkedList`'s `computeIfPresent()` method:

```
public V computeIfPresent(K key,
    ➤ BiFunction<? super K, ? super V, ? extends V> rf) {
    if (rf == null) {
        throw new NullPointerException();
    }

    Node<K,V> e;
    V oldValue;
    int hash = hash(key);
    e = getNode(hash, key);
    oldValue = e.value;

    if (e != null && oldValue != null) {
        V v = rf.apply(key, oldValue);

        if (v != null) {
            e.value = v;
            afterNodeAccess(e);
            return v;
        } else {
            removeNode(hash, key, null, false, true);
        }
    }
    return null;
}
```

What is the minimum number of tests required to achieve 100% branch coverage?

- A** 2
- B** 3
- C** 4
- D** 6

- 3.4** Consider the expression $(A \ \& \ B) \mid C$ with the following truth table:

Test case	A	B	C	$(A \ \& \ B) \mid C$
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T

Test case	A	B	C	(A & B) C
4	T	F	F	F
5	F	T	T	T
6	F	T	F	F
7	F	F	T	T
8	F	F	F	F

What test suite(s) achieve 100% MC/DC? The numbers correspond to the test case column in the truth table. Select all that apply.

A {2, 3, 4, 6}

B {2, 4, 5, 6}

C {1, 3, 4, 6}

D {3, 4, 5, 8}

3.5 Draw the truth table for the expression $A \ \&\& \ (A \ || \ B)$.

Is it possible to achieve MC/DC coverage for this expression? Why or why not? What would you tell the developer who wrote this expression?

3.6 Consider the following method:

```
public String sameEnds(String string) {
    int length = string.length();
    int half = length / 2;

    String left = "";
    String right = "";

    int size = 0;
    for (int i = 0; i < half; i++) {
        left = left + string.charAt(i);
        right = string.charAt(length - 1 - i) + right;

        if (left.equals(right)) {
            size = left.length();
        }
    }

    return string.substring(0, size);
}
```

Which of the following statements is *not* correct?

A It is possible to devise a single test case that achieves 100% line coverage and 100% decision coverage.

B It is possible to devise a single test case that achieves 100% line coverage and 100% (basic) condition coverage.

- C** It is possible to devise a single test case that achieves 100% line coverage and 100% decision + condition coverage.
 - D** It is possible to devise a single test case that achieves 100% line coverage and 100% path coverage.
- 3.7** Which of the following statements concerning the subsumption relations between test adequacy criteria is true?
 - A** MC/DC subsumes statement coverage.
 - B** Statement coverage subsumes branch coverage.
 - C** Branch coverage subsumes path coverage.
 - D** Basic condition coverage subsumes branch coverage.
- 3.8** A test suite satisfies the loop boundary adequacy criterion if for every loop L:
 - A** Test cases iterate L zero times, once, and more than once.
 - B** Test cases iterate L once and more than once.
 - C** Test cases iterate L zero times and one time.
 - D** Test cases iterate L zero times, once, more than once, and N, where N is the maximum number of iterations.
- 3.9** Which of the following statements is *correct* about the relationship between specification-based testing and structural testing?
 - A** A testing process should prioritize structural testing because it's cheaper yet highly effective (maybe even more effective than specification-based testing).
 - B** Specification-based testing can only be effectively performed when we have proper models of the program under test. A simple user story is not enough.
 - C** Boundary analysis can only be done if testers have access to the source code, and thus it should be considered a structural testing technique.
 - D** None of the other answers is true.

Summary

- Structural testing uses the source code to augment the test suite engineered via specification-based testing.
- The overall idea of structural testing is to analyze which parts of the code are not yet covered and reflect on whether they should be covered or not.
- Some coverage criteria are less rigorous and therefore less expensive (for example, line coverage). Others are more rigorous but also more expensive (such as MC/DC coverage). As a developer, you have to decide which criteria to use.
- Code coverage should not be used as a number to be achieved. Rather, coverage tools should be used to support developers in performing structural testing (that is, understanding what parts are not covered and why).
- Mutation testing ensures that our test suite is strong enough: in other words, that it can catch as many bugs as possible.